

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.(BL3)

Assessment Tool Weightage: 35%

QUESTIONS: 3

Duration : 1 Hour
Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I JAYADEEP.J(192511304) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

Constraints:

- i. D1 cannot work Night shift
- ii. D2 must work before D3 (Morning < Afternoon < Night)
- iii. Only one doctor per shift
- iv. D3 cannot work Morning
- v. All doctors must be assigned exactly one shift

Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

Constraints:

- i. Movement allowed: Up, Down, Left, Right
- ii. Cost of each move = 1
- iii. Cannot pass through obstacles
- iv. Use Manhattan Distance heuristic

Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

- a) Analyze all the given constraints and explain how they affect the robot's decision-making process.

- b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.
- c) Demonstrate how the robot applies AI concepts such as:
- Intelligent agents
 - Rationality
 - Environment type
- d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

ANSWER SHEET

Artificial Intelligence (CSA17) – CO1

Question 1: Doctor Shift Scheduling using Backtracking

Formulation as a CSP

- **Variables:** D1, D2, D3 (the shift of each doctor).
- **Domain:** {Morning (M), Afternoon (A), Night (N)}, ordered $M < A < N$.
- **Constraints:** (1) $D1 \neq N$; (2) $\text{shift}(D2) < \text{shift}(D3)$; (3) all three shifts are different (one doctor per shift); (4) $D3 \neq M$; (5) every doctor gets exactly one shift, which together with (3) means the assignment is a permutation of M, A, N.

Applying the unary constraints first shrinks the domains: $D1 = \{M, A\}$ and $D3 = \{A, N\}$, while D2 stays $\{M, A, N\}$. I take the variables in the order D1, D2, D3 and values in the order M, A, N.

Backtracking trace

Step	Assignment tried	Constraint check / result
1	D1 = M	Allowed by $D1 \neq N$. Consistent so far, go deeper.
2	D1 = M, D2 = M	Violates one doctor per shift (M already used by D1). Reject, try the next value.
3	D1 = M, D2 = A	No clash with D1. Constraint 2 is not yet testable (D3 unassigned). Consistent, go deeper.
4	D1 = M, D2 = A, D3 = M	$D3 \neq M$ is violated, and M is already used. Reject.
5	D1 = M, D2 = A, D3 = A	A is already used by D2 (one doctor per shift), and D2 would not be before D3. Reject.
6	D1 = M, D2 = A, D3 = N	$D1 \neq N$ ✓, D2 (A) before D3 (N) ✓, $D3 \neq M$ ✓, shifts all different ✓, all doctors assigned ✓. Complete assignment, so it is a solution.

Final valid schedule: **D1 – Morning, D2 – Afternoon, D3 – Night.**

Backtracking stops at the first valid assignment. If we kept going, we would find the only other valid schedule, D2 – Morning, D1 – Afternoon, D3 – Night ($D1 = A, D2 = M, D3 = N$), which also satisfies every constraint. Any schedule with D3 in the Morning, D1 at Night, or D2 after D3 fails, so these two are the only ones that survive.

Question 2: Robot Navigation on a 5×5 Grid

Note: the grid layout is not included in the question paper, so I have assumed the layout below. The method is identical for any other layout; only the numbers in the trace change.

Notation: cells are (row, column), with (1,1) in the top-left. S = (1,1), G = (5,5) and # marks an obstacle.

S	.	.	.	.
---	---	---	---	---

.	#	#	.	.
.	.	.	#	.
.	#	.	.	.
.	.	.	.	G

Obstacles: (2,2), (2,3), (3,4), (4,2). Moves are Up, Down, Left, Right, each costing 1, and obstacles cannot be entered.

How the search is set up

Plain BFS uses a FIFO queue and ignores any heuristic, but the question asks for a priority queue and Manhattan distance, which is really the best-first (A*) version of the same layered expansion. So I order the queue by $f(n) = g(n) + h(n)$, where g is the number of moves so far and $h(n) = |\text{row} - 5| + |\text{col} - 5|$ is the Manhattan distance to G. Because every move costs 1 and the Manhattan distance never overestimates in a grid with 4-directional moves, h is admissible and consistent, so the result is optimal. Ties in f are broken by smaller h , and neighbours are generated in the order Up, Down, Left, Right.

$h(S) = 4 + 4 = 8$, so $f(S) = 8$.

Step-by-step node expansion

Step	Expand (g, h, f)	Neighbours added (g, h, f)	Priority queue after step (cell: f)
1	(1,1) 0, 8, 8	(2,1) 1,7,8 (1,2) 1,7,8	(1,2):8, (2,1):8
2	(2,1) 1, 7, 8	(3,1) 2,6,8 [(2,2) is an obstacle; (1,1) already closed]	(3,1):8, (1,2):8
3	(3,1) 2, 6, 8	(4,1) 3,5,8 (3,2) 3,5,8	(3,2):8, (4,1):8, (1,2):8
4	(4,1) 3, 5, 8	(5,1) 4,4,8 [(4,2) is an obstacle]	(5,1):8, (3,2):8, (1,2):8
5	(5,1) 4, 4, 8	(5,2) 5,3,8	(5,2):8, (3,2):8, (1,2):8
6	(5,2) 5, 3, 8	(5,3) 6,2,8 [(4,2) is an obstacle]	(5,3):8, (3,2):8, (1,2):8
7	(5,3) 6, 2, 8	(4,3) 7,3,10 (5,4) 7,1,8	(5,4):8, (3,2):8, (1,2):8, (4,3):10
8	(5,4) 7, 1, 8	(4,4) 8,2,10 (5,5) 8,0,8	(5,5):8, (3,2):8, (1,2):8, (4,4):10, (4,3):10
9	(5,5) 8, 0, 8	Goal test is true, stop	—

Because (3,2) and (1,2) have the same f -value of 8, they are sitting in the queue but are never needed, since the goal comes out first on the tie-break (smaller h). Nodes with $f = 10$ are never expanded, which shows how the heuristic prunes detours.

Optimal path: (1,1) → (2,1) → (3,1) → (4,1) → (5,1) → (5,2) → (5,3) → (5,4) → (5,5), i.e. Down, Down, Down, Down, Right, Right, Right, Right.

Total cost = 8, the same as the Manhattan distance from S to G, so no shorter path can exist (8 moves is the minimum possible on a 5×5 grid between opposite corners). Nine nodes were expanded in total.

Question 3: Autonomous Rescue Robot

a) Effect of the constraints on decision-making

Constraint	Effect on the robot's decisions
Moves limited to up, down, left, right	The action set is small (at most 4 actions per cell), so branching factor is low and no diagonal shortcuts are available. Distances are Manhattan-style.
Blocked cells (obstacles)	Some actions are not available in some states. The robot must detect these and remove them from its options, and it may need to backtrack from dead ends.
Limited sensing (adjacent cells only)	The environment is partially observable. The robot cannot plan the whole path in advance, because it does not know the layout beyond its neighbours. It has to build a map as it moves, which makes this an online search problem.
Move cost 1, risky cell entry +2	Entering a risky cell costs 3 rather than 1, so the shortest path in steps is not always the cheapest. The robot has to compare cost, not just length, which rules out plain BFS.
Minimise total cost with safe navigation	There are two objectives, cost and safety. The robot should avoid risky cells unless the saving is worth it, and it must never take an action that could leave it trapped.
Dynamic knowledge update	Every new observation (a newly found obstacle or risky cell) can make the current plan invalid, so the robot must update its map and re-plan.

Taken together, the constraints show that the robot cannot just follow a fixed route. It must keep updating its belief about the map, evaluate paths by cost, and be ready to change course. A small example: suppose one route to G is 5 moves with one risky cell, giving $5 + 2 = 7$, and a safe route is 6 moves, costing 6. The longer route is actually cheaper, so the robot picks it.

b) Solution approach

I would use an **online search agent built on Uniform Cost Search** (re-planning with UCS over the map the robot knows so far). The steps are:

- **Initialise:** store the start S as the current cell, create an empty map (cell $\rightarrow$ free / blocked / risky / unknown) and set the cost-so-far to 0.
- **Sense:** read the four adjacent cells and add them to the map with their status (free, blocked or risky).
- **Plan:** run UCS from the current cell over the known map, treating unknown cells as passable at normal cost (an optimistic assumption). Step cost is 1 for a normal cell and $1 + 2 = 3$ for a risky cell; blocked cells are not expanded. UCS picks the node with the lowest $g(n)$ from a priority queue.
- **Act:** carry out only the first move of the cheapest path found.
- **Update and check:** sense again. If a new obstacle or risky cell lies on the planned route, discard the plan and re-run UCS from the new position. Otherwise continue along the plan.
- **Avoid loops and dead ends:** keep a record of visited cells and the cost of reaching them. If

every neighbour is blocked or already explored, backtrack to the nearest cell that still has an unexplored option.

- **Terminate:** stop when the current cell is G (the survivor), and record the total cost.

c) AI concepts demonstrated

Intelligent agent. The robot fits the agent definition perceiving through sensors and acting through actuators. Its percepts are the contents of adjacent cells and its current position; its actions are the four moves. Because it keeps an internal map, remembers visited cells and works towards a goal while weighing costs, it behaves as a model-based, goal-based and utility-based agent.

Rationality. A rational agent chooses the action expected to maximise its performance measure given what it knows. Here the performance measure is reaching the survivor at the lowest total cost with safe navigation. The robot is rational because at every step it chooses the move on the cheapest known path, avoids risky cells unless the saving justifies them, and re-plans when it learns something new. Rationality is not the same as omniscience: it can still choose the best action it knows of, even if later information shows it was not the best in hindsight.

Environment type. The environment is partially observable (sees only adjacent cells), single agent, sequential (earlier moves change later costs), discrete (grid cells and steps), and unknown at the start. I would treat it as static during the search, apart from the changes in the robot's knowledge, though in a real collapsing building it could be dynamic, which only strengthens the case for re-planning. Moves are assumed deterministic. These properties are what make the online search approach necessary.

d) Why this approach is the most suitable

- **Costs are not uniform.** BFS and plain DFS ignore step costs, so they cannot handle the +2 penalty. UCS is built for exactly this and guarantees the cheapest path on the known map.
- **Optimality and completeness.** UCS is complete and optimal when all step costs are positive, which is the case here (minimum cost 1).
- **The map is not known in advance.** Offline planning (solve fully, then act) is impossible with local sensing. An online agent that plans, moves, senses and re-plans is the only realistic way of working, and it also supports the requirement to update knowledge dynamically.
- **Safety.** Giving risky cells a higher cost builds safety straight into the search, so no separate mechanism is needed. A very high penalty could be used if cells must be avoided completely.
- **No heuristic needed.** A* could be faster, but it needs a good admissible heuristic, and with unknown obstacles and risky zones the Manhattan distance can be misleading. UCS makes no such assumption, and the grid is small enough that the extra nodes expanded are not a concern.

The trade-off is that UCS may expand more nodes than an informed search and may re-plan often. For a small building grid this cost is acceptable compared with the benefit of guaranteed cost-

optimal and safe behaviour.